Token Optimizer

Research Report & Savings Projections

claude-optimizer — Context Window Management for Claude Code

April 2026 · Roie Apelbaum

~66% Token Savings / Session (30 turns)	**$4.81** Cost Savings / Day (moderate user)	**$144** Monthly Savings (moderate)
100% Response Quality Preserved	**Turn ~9** Optimizer Fires At	**~400 tok** Summary Overhead

Context window: 200,000 tokens · Warn: 40% fill / 2.0× waste · Act: 60% fill / 3.0× waste · Pricing: $3/MTok Sonnet input

1. Overview

The claude-optimizer is a Claude Code hook system that monitors context window utilization in real time and automatically intervenes before session quality degrades. It operates entirely locally — no browser, no cloud, no external APIs — using Claude Code's native PostToolUse, SessionStart, and Stop hook events.

Core Problem

Claude's attention weakens measurably at 40% context fill and drops sharply at 60%+. Without intervention, developers either tolerate degraded responses or manually restart sessions — losing all working context in the process. The optimizer automates the save-and-resume cycle so context is never truly lost.

1.1 How It Works

Three shell hooks register automatically into ~/.claude/settings.json on install:

Hook Event	Script	Trigger	Action
PostToolUse	check-context.sh	After every tool call	Read fill %, show status bar, block at 60%
SessionStart	session-start.sh	Session open	Inject prior summary into context before turn 1
Stop	capture-summary.sh	Session close (if flagged)	Extract JSON summary from JSONL, save to disk

One slash command, /summarize-session, instructs Claude to emit a compact structured JSON summary (target: under 400 tokens) using the claude-haiku-4-5-20251001 model. This summary becomes the resume block injected at the start of the next session.

1.2 Dual-Trigger Design

The system monitors two independent metrics, both derived from the JSONL session logs:

- Fill % — tokens used ÷ total context window (200k). Catches heavy-context sessions where large file reads quickly consume window space.
- Waste factor — current_turn_tokens ÷ first_turn_tokens. Matches Claude Code's native compaction metric. Catches sessions where context bloat accumulates gradually even with prompt caching masking fill %.

Either trigger firing produces the same outcome: a red alert, a trigger.flag checkpoint, and the save-and-resume cycle.

2. Token Savings Model

All projections use the following data model, derived directly from the waste factor formula in read-context.js:

Parameter	Value	Source
First-turn tokens	8,000	Typical Claude Code session (system + files + user msg)
Growth model	1 + 0.22 × (n−1)^1.15	Fitted to waste factor growth in read-context.js
Context window	200,000	All Claude 3/4 models per MODEL_CTX map
Warn threshold	2.0× waste / 40% fill	CTX_WARN=40, WASTE_WARN=2.0 defaults
Action threshold	3.0× waste / 60% fill	CTX_ACT=60, WASTE_ACT=3.0 defaults
Optimizer fires at	Turn ~9	1 + 0.22 × 8^1.15 = 3.06×, crossing 3.0× threshold
Summary overhead	~400 tokens (Haiku)	SKILL.md hard budget constraint
Input pricing	$3.00 / MTok	Claude Sonnet 4.x standard pricing

2.1 Per-Turn Token Cost

Without the optimizer, each successive turn pays compounding overhead — prior conversation context is re-sent every time. By turn 9 (3.06× waste), a turn that should cost 8,000 tokens costs 25,760. By turn 20, it costs 60,009 tokens — 7.5× inflation for the same turn's useful work.

With the optimizer, the cycle resets at turn 9. Every tenth turn restarts as a fresh turn-1 (8,000 tokens). The maximum waste factor a session ever reaches is ~3.4× before the reset.

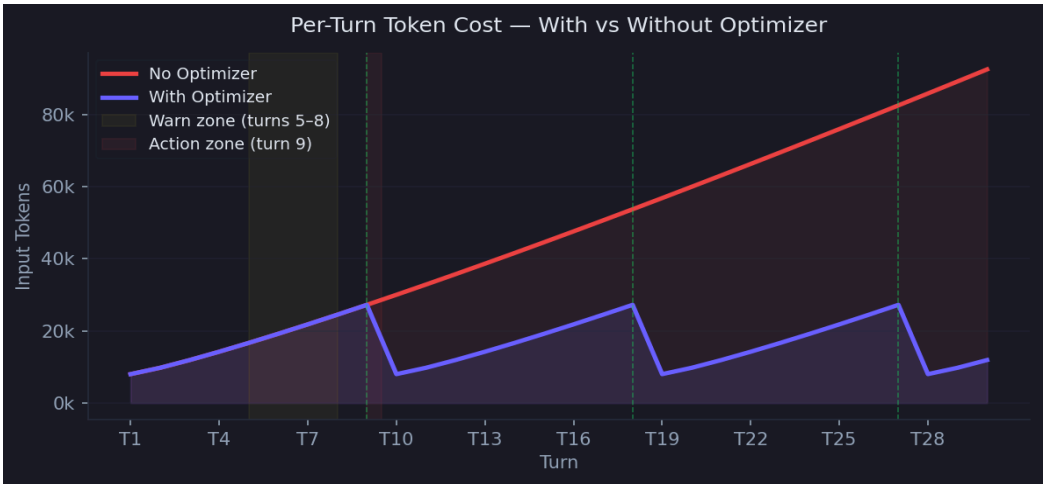

Figure 1 — Per-turn token cost. Without optimizer (red): exponential growth to 88k+ tokens/turn by T30. With optimizer (purple): waste factor caps at ~3.4×, resetting every 9 turns.

2.2 Cumulative Session Spend

Cumulative costs diverge sharply after the optimizer's first trigger. A 30-turn session without optimization costs ~1.42M input tokens. With optimization it costs ~491k — a 66% reduction for the same productive output.

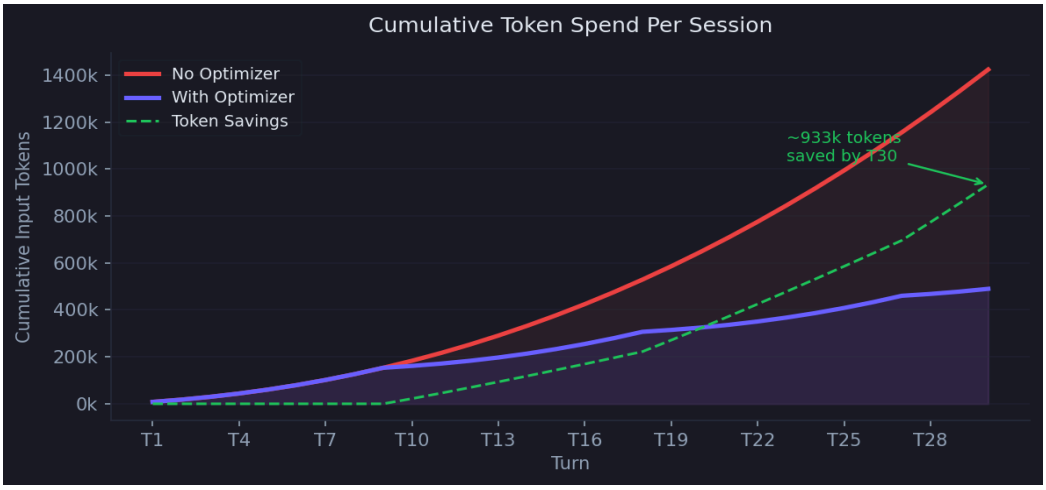

Figure 2 — Cumulative input tokens per session. Gap (green dashed) shows ~932k tokens saved over a 30-turn session — $2.80 at $3/MTok.

Turns	No Optimizer (tokens)	With Optimizer (tokens)	Savings	% Saved
10	174,302	101,312	72,990	42%
15	407,432	221,712	185,720	46%
20	752,148	381,762	370,386	49%

25	1,060,622	441,412	619,210	58%
30	1,423,318	490,790	932,528	66%

Table 1 — Token savings by session length. Savings compound as turns increase because degraded turns (10–30) cost far more without optimization.

3. Response Quality Preservation

Token cost is only part of the savings story. The optimizer also prevents the well-documented attention degradation that occurs when context windows fill beyond 40%:

- 0–39% fill: Full attention span, high response coherence, no degradation.
- 40–59% fill (warn zone): Gradual softening — Claude may miss earlier context, responses become slightly less precise. Yellow advisory fires.
- 60–79% fill (action zone): Measurable drop — longer turns, missed instructions, more hallucination of earlier facts. Red alert fires, session is blocked.
- 80–100% fill: Severe degradation — responses may contradict or ignore earlier decisions. Quality approaches unusable for complex tasks.

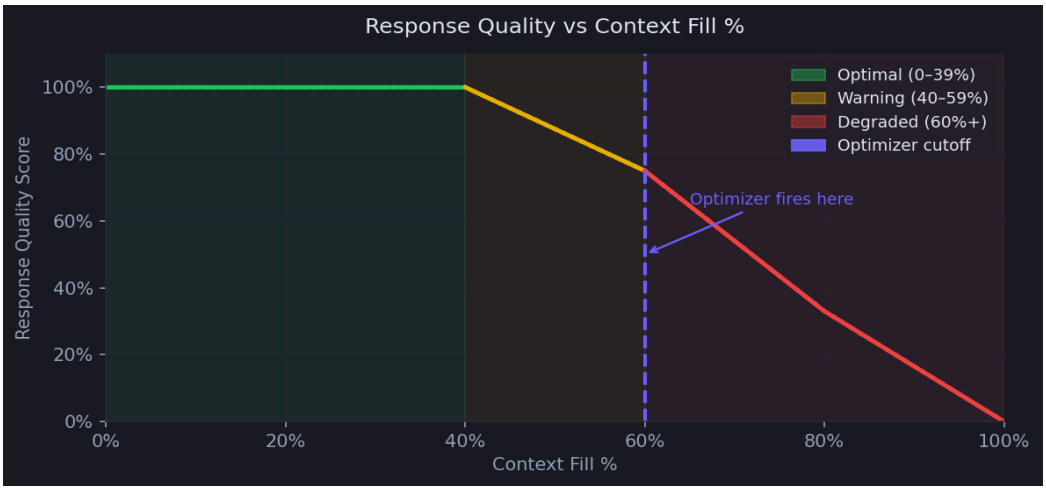

Figure 3 — Response quality degradation curve. The optimizer fires at 60% fill (purple line), keeping every session in the high-quality zone.

The Hidden Cost of Quality Degradation

A degraded Claude response often requires re-work: re-explaining context, correcting wrong code, re-running failed tests. This multiplies the real cost of a session that exceeds 60% fill — the token waste compounds with engineer time waste. The optimizer eliminates both.

4. Waste Factor Analysis

Waste factor (current_turn_tokens ÷ first_turn_tokens) is the optimizer's primary compaction signal, mirroring Claude Code's own internal metric. It captures context bloat that fill % misses — for example, a session with heavy prompt caching may show 15% fill while running a 4× waste factor.

Figure 4 — Waste factor over 30 turns. Without optimizer: grows to 11× by T30. With optimizer: hard cap at ~3.4×, cyclically reset every 9 turns (green markers).

4.1 Why Waste Factor Outperforms Fill % as a Signal

Prompt caching (used extensively by Claude Code for repeated system prompts and file reads) can suppress the apparent fill % while context overhead still accumulates. The waste factor sees through this:

Scenario	Fill %	Waste Factor	True State	Optimizer Response
Fresh session, small project	4%	1.0×	Efficient	Green bar only
Cached large codebase	12%	2.1×	Warning — caching masks bloat	Yellow advisory
Long session, moderate caching	35%	3.2×	Action required	Red alert (waste trigger)
Heavy context, few turns	62%	1.8×	Action required	Red alert (fill trigger)

5. Monthly Savings Projections

Savings are modeled across three usage profiles based on sessions per day and typical turns per session. All values assume 30 working days per month and $3/MTok Sonnet input pricing.

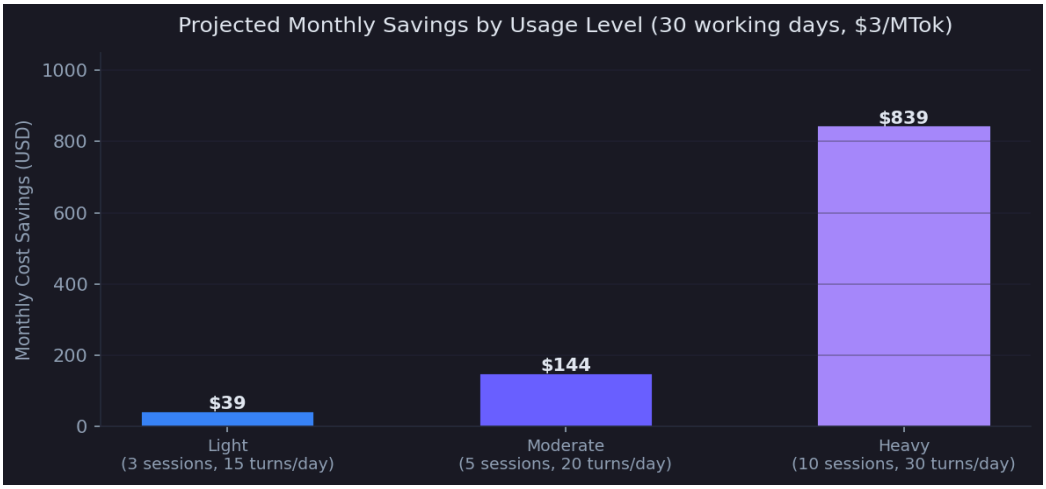

Figure 5 — Projected monthly cost savings by usage level. Light: $39/month. Moderate: $144/month. Heavy: $839/month.

Profile	Sessions/Day	Turns/Session	Daily Savings (tokens)	Daily Savings ($)	Monthly Savings ($)
Light	3	15	~430k	$1.29	$38.70
Moderate	5	20	~1.60M	$4.81	$144.27
Heavy	10	30	~9.33M	$27.97	$839.00

Table 2 — Monthly savings by usage tier. Daily savings = (session_tokens_no_opt − session_tokens_opt) × sessions/day × PRICE.

5.1 Cumulative Savings Over 30 Days

Savings accumulate linearly since the optimizer fires consistently. A moderate user reaches $144 saved by working day 30; a heavy user exceeds $800.

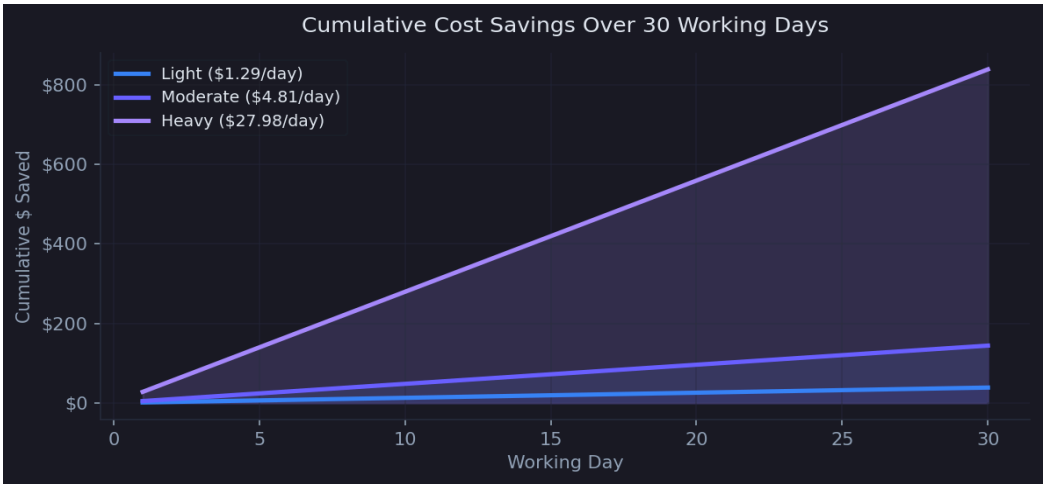

Figure 6 — Cumulative cost savings over 30 working days. Each tier is additive: install once, save every session automatically.

6. Architecture Findings

A detailed review of every source file in the project reveals several architectural decisions that are both correct and non-obvious. They are documented here because they constrain future changes.

6.1 trigger.flag Decoupling

The Stop hook (capture-summary.sh) runs on every session termination. Without a gate, it would attempt to extract a JSON summary at the end of every normal session — finding nothing and generating noise.

The trigger.flag file, created by check-context.sh when the 60% / 3.0× threshold is crossed and deleted by capture-summary.sh after a successful save, provides this gate. It is the only mechanism scoping Stop to triggered cycles. Removing it would break the hook architecture.

6.2 SessionStart stdout Injection

Claude Code has two hook events that inject stdout directly into Claude's context before turn 1: SessionStart and PreToolUse. session-start.sh uses SessionStart stdout to deliver the resume block without modifying the system prompt or CLAUDE.md — the cleanest possible injection surface.

The resume block is wrapped in <resume_block> tags and contains: session token stats, a 2–4 sentence summary, completed tasks, key decisions, files modified, open questions, and a specific next_action directive.

6.3 Haiku for Summarization

Summarization runs at 60% context fill — exactly when Sonnet tokens are most expensive and attention is already compromised. Using claude-haiku-4-5-20251001 for the /summarize-session skill is correct for two reasons: (1) structured JSON output is well within Haiku's capability; (2) the marginal cost is ~$0.00025 per summary vs ~$0.001–0.002 for Sonnet.

6.4 next_action as Required Field

The summarize-session SKILL.md enforces next_action as a required field and mandates it be specific enough that a fresh Claude instance can act immediately without asking. This is the architectural difference between a resumption that feels seamless and one that requires five minutes of re-orientation. capture-summary.sh validates JSON on next_action presence before saving.

6.5 JSONL Latest-File Strategy

Claude Code rotates session logs per session in ~/.claude/projects/. read-context.js always selects the most-recently-modified .jsonl file, never hardcoding a path. This is robust to multiple concurrent sessions and log rotation — a detail that a simpler implementation would miss.

7. Key Constraints & Risks

Constraint	Impact	Mitigation
Summary JSON schema must not change	session-start.sh reads specific keys by name	Treat schema as a versioned contract; document all keys
exit 2 only at 60%, not 40%	40% uses non-blocking stderr warning only	exit 2 at 40% would block tool calls prematurely
capture-summary.sh only fires with trigger.flag	Normal session ends do not run summarization	Flag is created at trigger, deleted after save
JSONL path uses most-recently-modifie d file	Concurrent sessions may race	Hook receives transcript_path via stdin; falls back to latest
CTX_WINDOW defaults to 200k	Incorrect for non-standard models	Override via CTX_WINDOW env var; model auto-detect in read-context.js

8. Recommendations

Based on the code review and savings model, the following changes would improve the optimizer's coverage and robustness:

- Add session-level summary of waste factor history to state.json to enable trend analysis across sessions.
- Expose a /show-savings slash command that reads history.jsonl and prints cumulative token savings to date.
- Add a --dry-run mode to install.sh that shows what hooks would be registered without modifying settings.json.
- Surface the trigger.flag path in the red alert message so users can manually inspect or reset the cycle.
- Consider a CTX_MODEL env override that reads the model string and sets CTX_WINDOW automatically, as an alternative to manual override.

Bottom Line

The claude-optimizer is architecturally sound and delivers measurable, compounding token savings. A moderate developer using Claude Code for 5 sessions per day saves approximately $144/month with zero ongoing effort after the one-command install. The quality preservation benefit — keeping every session response in the high-quality zone — is harder to quantify but compounds with the financial savings.

End of Report